

## Indexes

An **index** is a data structure that speeds access to tuples of a relation given values of one or more attributes. Without an index, the DBMS must scan the entire table. Indexes are declared separately from the table and maintained automatically.

```
CREATE [UNIQUE] [CLUSTERED] INDEX name ON Table(col1, col2, ...);
DROP INDEX name;
```

| Modifier   | Effect                                                           |
|------------|------------------------------------------------------------------|
| UNIQUE     | Enforces no duplicate values in the indexed column(s).           |
| CLUSTERED  | Sorts the data rows to match index order; at most one per table. |
| USING HASH | Uses a hash structure instead of the default B+ tree.            |

**Trade-off.** An index speeds up **SELECT** but slows down **INSERT/UPDATE/DELETE** because the index must also be updated. Always weigh query speedup against write overhead.

| Classification         | Meaning                                                        |
|------------------------|----------------------------------------------------------------|
| <b>Primary</b> index   | Search key contains the relation's primary key.                |
| <b>Unique</b> index    | Search key is a superkey (guarantees uniqueness).              |
| <b>Secondary</b> index | Any other index; multiple allowed per table.                   |
| <b>Clustered</b>       | Index order matches disk order of data records.                |
| <b>Unclustered</b>     | Index order differs from data order; range scans are costlier. |

## B+ Tree Indexes (default in PostgreSQL)

The most common index structure. Supports both **equality** and **range** searches efficiently. Each node is at least 50% full; large index files are stored on disk and partially loaded into memory.

| Node type                  | Contents                                                                                                 |
|----------------------------|----------------------------------------------------------------------------------------------------------|
| <b>Non-leaf (internal)</b> | Keys and pointers to child nodes only.                                                                   |
| <b>Leaf</b>                | Actual data entries (or pointers to them), sorted by search key; leaves linked together for range scans. |

**Search.** Traverse from root, following pointers: go left of key  $k$  if value  $\leq k$ , right if value  $> k$ . Arrive at leaf; scan there.

### Insert.

1. Find the correct leaf  $L$  (traverse from root).
2. **If  $L$  has space:** insert and done.
3. **If  $L$  is full:** split  $L$  into  $L$  and  $L_2$ , redistribute entries evenly, *copy up* the middle key to the parent; insert pointer to  $L_2$  in parent (may propagate recursively to root).

### Delete.

1. Find the leaf  $L$  containing the entry (traverse from root).
2. **If  $L$  is still  $\geq$  half-full:** remove entry, done.
3. **If  $L$  underflows:** try to *redistribute* by borrowing from a sibling.
4. **If redistribution fails:** *merge*  $L$  with its sibling; delete the separating key from the parent (may propagate to root).

*Splitting leaf nodes **copies** the middle key up; splitting internal nodes **pushes** it up (key is removed from the node).*

## Hash-Based Indexes

Best for **equality** selections only; do **not** support range queries.

| Concept           | Meaning                                                                    |
|-------------------|----------------------------------------------------------------------------|
| Hash function $h$ | Maps a search-key value to a bucket number $0 \dots N-1$ .                 |
| Bucket            | Primary page + zero or more chained overflow pages; contains data entries. |
| Search            | Apply $h$ to the key; search only that bucket — no full scan needed.       |
| Insert            | Apply $h$ ; if no space in primary page, allocate an overflow page.        |
| Delete            | Apply $h$ ; locate and remove entry; free overflow page if now empty.      |

*Overflow pages degrade performance; choose  $h$  and bucket count carefully.*

Use Side A as a reference. For B+ tree questions, assume order  $d = 2$  (each node holds at most 4 entries / 5 pointers) unless stated otherwise. Schema: **Enrolled**(sid, cid, grade).

### Box 1 Declaring & Choosing Indexes (6 min)

(a) Write the SQL to create a **clustered** B+ tree index called **EnrolledGradeIdx** on the **grade** column of **Enrolled**.

(b) A classmate adds a hash index on **cid**. A common query is:

```
SELECT * FROM Enrolled WHERE cid > 'CSC343';
```

Will the hash index help? In one sentence, explain why or why not.

.....

.....

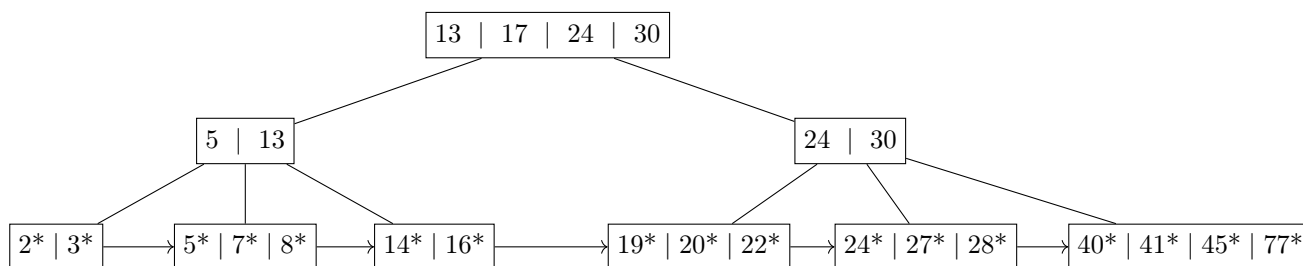
(c) Name **one** scenario where having many indexes on a table is *harmful*.

.....

.....

### Box 2 B+ Tree Search & Insert (10 min, discuss)

Consider the B+ tree below with order  $d = 2$  (max 4 keys per leaf, max 4 keys per internal node).



(a) What leaf do you reach when searching for key **20\***? Trace the path from the root.

.....

(b) Insert key **23\***. Does a split occur? Show the resulting leaf.

(c) Starting from the *original* tree, insert **8\*** (it is not yet present, add it to leaf [5\*|7\*] region). Pretend a leaf split occurs. Which key is *copied up* to the parent, and does the parent also split? Explain briefly.

.....  
.....

**Box 3 Hash Index (6 min)**

---

A hash index on `grade` uses  $h(\text{grade}) = \text{ASCII}(\text{grade}) \bmod 3$  and 4 slots per bucket page. Grades are single characters; ASCII: A=65, B=66, C=67, D=68, F=70.

(a) Into which buckets (0, 1, or 2) do grades A, B, C, D, F hash?

.....

(b) Suppose 5 students have grade A and 5 have grade B. Sketch (no need to draw precisely) whether bucket 2 needs an overflow page given 4 slots per primary page.

(c) Can this hash index efficiently answer `WHERE grade >= 'B'`? Why or why not?

.....  
.....